What is the extension of compiled java classes?

Flag

Previous

Next

Answer

.txt

.class

.js

.java

The correct answer is: **.class**

In Java, when you compile a Java source file (.java), the compiler generates a compiled class file with a `.class` extension. This file contains the bytecode that can be run on the Java Virtual Machine (JVM).

What will be the output of the following Java program?

```
class box
{
int width;
int height;
int length;
}
class mainclass
{
public static void main(String args [])
{
box obj = new box();
obj.width = 10;
obj.height = 2;
obj.length = 10;
int y = obj.width * obj.height * obj.length ;
System.out.print (y);
}
}
```

Flag

Previous

Next

Answer

12

200

00

400

The correct answer is: **200**

Here's the explanation:

In the given Java program, an object `obj` of class `box` is created. The properties `width`, `height`, and `length` of the object are assigned values 10, 2, and 10 respectively.

Then, the variable `y` is calculated by multiplying the values of `width`, `height`, and `length` together:

```
y = obj.width * obj.height * obj.length
y = 10 * 2 * 10
y = 200
```

So, the output of the program will be **200**.

What will be the output of the following Java program?

```
class box
{
int width;
int height;
int length;
}
class mainclass
{
public static void main(String args [])
{
box obj = new box();
obj.width = 10;
obj.height = 2;
obj.length = 10;
```

```
int y = obj.width * obj.height * obj.length ;
System.out.print (y);
}
}
```

Flag

Previous

Next

Answer

12

200

00

400

The correct answer is: **200**

Here's the step-by-step calculation:

1. `obj.width` is 10
2. `obj.height` is 2
3. `obj.length` is 10
4. `y` is calculated as `width * height * length` = 10 * 2 * 10 = 200

So, the output of the program will be **200**.

Note: The other options (12, 00, and 400) are incorrect because they do not match the result of the calculation.

Which of the following is not a segment of memory in java?

Flag

Previous

Next

Answer

Heap Segment

Stack Segment

Register Segment

Code Segment

The correct answer is: **Register Segment**

In Java, the segments of memory are:

1. **Heap Segment**: This is where objects are stored. It's a shared memory space where objects are dynamically allocated and deallocated.
2. **Stack Segment**: This is where primitive types and object references are stored. It's a region of memory where data is added and removed in a last-in, first-out (LIFO) order.
3. **Code Segment** (or Method Area): This is where the JVM stores the bytecode of the classes and methods.

However, **Register Segment** is not a memory segment in Java. In computer architecture, registers are small, high-speed memory locations within the CPU that store data temporarily while it's being processed. Java does not have direct access to CPU registers.

So, the correct answer is **Register Segment**.

If an expression contains double, int, float, long, then the whole expression will be promoted into which of these data types?

Flag

Previous

Next

Answer

double

long

float

int

Here's the answer:

The correct answer is: **double**

In Java, when an expression contains multiple data types, the expression is promoted to the data type that can hold the largest value. The order of promotion is as follows:

1. `int` is promoted to `long`
2. `long` is promoted to `float`
3. `float` is promoted to `double`

So, if an expression contains `double`, `int`, `float`, and `long`, the whole expression will be promoted to `double`, because `double` is the largest data type in the expression.

Here's an example:

Java

```java
double result = 10 + 20L + 30.5f + 40;
```

In this example, the `int` value `10` is promoted to `long`, then the `long` value `20L` is promoted to `float`, and finally the `float` value `30.5f` is promoted to `double`. The result is a `double` value.

What will be the output of the following Java code?
class A
{
final public int calculate(int a, int b) { return 1; }

```
}
class B extends A
{
public int calculate(int a, int b) { return 2; }
}
public class output
{
public static void main(String args [])
{
B object = new B();
System.out.print ("b is " + b.calculate (0, 1));
}
}
```

Flag

Previous

Next

Answer

Compilation Error

An exception is thrown at runtime

b is : 2

b is : 1

The correct answer is: **Compilation Error**

The reason is that the `calculate` method in class `A` is declared as `final`, which means it cannot be overridden in any subclass. However, class `B` is trying to override the `calculate` method, which is not allowed.

When the Java compiler encounters this code, it will throw a compilation error, stating that the `calculate` method in class `B` cannot override the `calculate` method in class `A` because it is declared as `final`.

So, the output will not be any of the other options ("An exception is thrown at runtime", "b is : 2", or "b is : 1"), but rather a compilation error.

What will be the output of the following Java code?

```
class output
{
```

```
public static void main(String args [])
{
String c = "Hello i love java";
boolean var ;
var = c.startsWith ("hello");
System.out.println ( var );
}
}
```

Flag

Previous

Next

Answer

1

0

true

false

The correct answer is: **false**

Here's why:

The `startsWith()` method in Java is case-sensitive, meaning it treats uppercase and lowercase letters as distinct characters.

In the given code, the string `c` is "Hello i love java" (with a capital "H"), but the argument passed to `startsWith()` is "hello" (with a lowercase "h").

Since the case doesn't match, the `startsWith()` method returns `false`, indicating that the string `c` does not start with the specified prefix "hello".

So, the output of the program will be **false**.

What will be the output of the following Java code?

```
class increment {
public static void main(String args [])
{
int g = 3;
System.out.print (++g * 8);
}
```

```
}
```

Flag

Previous

Next

Answer

25

24

33

32

The correct answer is: **32**

Here's the explanation:

In the given code, the variable `g` is initialized with the value `3`. The expression `++g` is a prefix increment operator, which increments the value of `g` by `1` before evaluating the expression.

So, the expression `++g * 8` can be broken down as follows:

1. `g` is incremented to `4` (due to the prefix increment operator `++g`)
2. The value of `g` (now `4`) is multiplied by `8`

Therefore, the output of the program will be `4 * 8 = 32`.

Which of these are selection statements in Java?

Flag

Previous

Next

Answer

for()

if()

break

continue

The correct answer is: **if()**

In Java, selection statements are used to control the flow of a program based on conditions. The `if` statement is a selection statement that evaluates a boolean expression and executes a block of code if the expression is true.

The other options are not selection statements:

- `for()` is a looping statement, used to execute a block of code repeatedly.
- `break` is a control statement, used to exit a loop or switch statement.
- `continue` is a control statement, used to skip the current iteration of a loop.

Other selection statements in Java include:

- `if-else`
- `if-else-if`
- `switch`

These statements allow you to make decisions in your code based on conditions, and execute different blocks of code accordingly.

Which of the following package is used for text formatting in Java programming language?

Flag

Previous

Next

Answer

java.io

java.text

java.awt

java.awt.text

The correct answer is: **java.text**

The `java.text` package in Java provides classes for text formatting, such as:

- `DateFormat` for formatting dates and times
- `NumberFormat` for formatting numbers and currencies
- `MessageFormat` for formatting messages with placeholders
- `ChoiceFormat` for formatting choices and ranges

This package provides a way to format text in a locale-independent manner, making it useful for internationalization and localization.

The other options are not primarily used for text formatting:

- `java.io` is for input/output operations
- `java.awt` is for Abstract Window Toolkit (AWT) graphics and UI components
- `java.awt.text` is a subpackage of `java.awt` for text components and editors, but not specifically for text formatting.

What will be the output of the following Java program?

```
class Alligator
{
public static void main(String[] args )
{
int []x[] = {{1,2}, {3,4,5}, {6,7,8,9}};
int [][]y = x;
```

```
System.out.println (y[2][1]);
}
}
```

Flag

Previous

Next

Answer

2

7

Compilation Error

3

The correct answer is: **7**

Here's the explanation:

In the given Java program, a 2D array `x` is declared and initialized with three sub-arrays:

- `x[0] = {1, 2}`
- `x[1] = {3, 4, 5}`
- `x[2] = {6, 7, 8, 9}`

Then, a new 2D array `y` is declared and assigned the value of `x`. This means that `y` now points to the same memory location as `x`.

The program then prints the value of `y[2][1]`, which corresponds to the second element of the third sub-array. In this case, `y[2][1]` is equal to `7`.

So, the output of the program will be **7**.

What will be the output of the following Java program, if we run as “java main_arguments 1 2 3”?

```
class main_arguments
{
public static void main(String [] args )
{
String [][] argument = new String[2][2];
int x;
argument[0] = args ;
```

```
x = argument[0].length;
for (int y = 0; y < x; y++)
System.out.print (" " + argument[0][y]);
}
}
```

Flag

Previous

Next

Answer

1 1

1 0

1 2 3

1 0 3

The correct answer is: **1 2 3**

Here's the explanation:

When you run the program as "java main_arguments 1 2 3", the command-line arguments are passed to the `args` array in the `main` method.

In this program, the `args` array is assigned to `argument[0]`, which is a 2D array. However, since `args` is a 1D array, it can be assigned to a single row of the 2D array.

The program then prints the elements of `argument[0]`, which are the command-line arguments "1", "2", and "3".

So, the output of the program will be **1 2 3**.

Note that the length of `argument[0]` is 3, which is the number of command-line arguments passed. The variable `x` is assigned this length, and the `for` loop iterates over the elements of `argument[0]`, printing each one.

Which of these coding types is used for data type characters in Java?

Flag

Previous

Next

Answer

None of the mentioned

UNICODE

ISO-LATIN-1

ASCII

The correct answer is: **UNICODE**

In Java, the `char` data type is used to represent single characters, and it uses Unicode encoding. Unicode is a 16-bit character encoding standard that supports a wide range of languages and symbols.

Java's `char` data type is a 16-bit unsigned integer that can represent any Unicode character, including letters, digits, punctuation marks, and special characters.

The other options are not correct:

- **ASCII** is a 7-bit character encoding standard that is a subset of Unicode, but it is not the default encoding used by Java for `char` data type.
- **ISO-LATIN-1** is an 8-bit character encoding standard that is also a subset of Unicode, but it is not the default encoding used by Java for `char` data type.
- **None of the mentioned** is incorrect because Unicode is indeed the encoding used by Java for `char` data type.

Which environment variable is used to set the java path?

Flag

Previous

Next

Answer

CLASSPATH

JAVA

MAVEN_HOME

JAVA_HOME

The correct answer is: **JAVA_HOME**

The `JAVA_HOME` environment variable is used to specify the location of the Java Development Kit (JDK) or Java Runtime Environment (JRE) on your system. It is used to set the Java path, which is the location where Java is installed.

The other options are not correct:

- **CLASSPATH** is an environment variable that specifies the location of user-defined classes and packages, but it is not used to set the Java path.
- **JAVA** is not a valid environment variable for setting the Java path.
- **MAVEN_HOME** is an environment variable used to specify the location of Apache Maven, a build automation tool for Java projects, but it is not related to setting the Java path.

Note: `JAVA_HOME` should point to the JDK or JRE installation directory, and it is used by various Java tools and applications to locate the Java installation.

hat is Truncation in Java?

Flag

Previous

Next

Answer

Floating-point value assigned to a Floating type

Floating-point value assigned to an integer type

Integer value assigned to floating type

Integer value assigned to floating type

The correct answer is: **Floating-point value assigned to an integer type**

Truncation in Java refers to the loss of precision that occurs when a floating-point value (such as a `double` or `float`) is assigned to an integer type (such as an `int` or `long`). This happens because integer types cannot represent fractional values, so the fractional part of the floating-point value is discarded, or "truncated".

For example:

Java

```java
double d = 3.7;
int i = (int) d; // truncation occurs here
System.out.println(i); // prints 3
```

In this example, the fractional part of the `double` value `3.7` is truncated when it is assigned to an `int`, resulting in a loss of precision.

The other options are not correct:

- **Floating-point value assigned to a Floating type**: This is not truncation, as the value is already a floating-point type.
- **Integer value assigned to floating type**: This is not truncation, as the integer value can be exactly represented by the floating-point type.
- **Integer value assigned to floating type**: This is the same as the previous option, and is not truncation.

What will be the output of the following Java code snippet?

```
class abc
{
public static void main(String args [])
{
if( args.length >0)
System.out.println ( args.length );
}
}
```

Flag

Previous

Next

Answer

The snippet compiles and runs but does not print anything

The snippet compiles, runs and prints 1

The snippet compiles, runs and prints 0

The snippet does not compile

The correct answer is: **The snippet compiles and runs but does not print anything**

Here's why:

- The code checks if the length of the `args` array is greater than 0.
- If the length is greater than 0, it prints the length of the `args` array.
- However, in this case, the `args` array is empty, because no command-line arguments are passed to the program.
- Therefore, the condition `args.length > 0` is false, and the `println` statement is not executed.
- As a result, the program compiles and runs successfully, but does not print anything.

Note that if you run the program with command-line arguments, like `java abc hello world`, the output would be `2`, because the `args` array would contain two elements: `hello` and `world`.

What will be the output of the following Java program?

class c

{

public void main(String[] args)

{

System.out.println ("Hello" + args [0]);

}

}

Flag

Previous

Next

Answer

Hello c

Hello world

Hello

Runtime Error

The correct answer is: **Runtime Error**

Here's why:

- The `main` method in Java should be declared as `public static void main(String[] args)`, but in this program, it's declared as `public void main(String[] args)`, missing the `static` keyword.
- Additionally, the program tries to access the first command-line argument using `args[0]`, but it doesn't check if any arguments are provided.
- If no arguments are provided, `args[0]` will throw an `ArrayIndexOutOfBoundsException`.

To fix this, the program should be modified to:

Java

```java
public class c {
    public static void main(String[] args) {
```

```
        if (args.length > 0) {
            System.out.println("Hello " + args[0]);
        } else {
            System.out.println("Hello");
        }
    }
}
```

With this modification, the program will print "Hello" if no arguments are provided, or "Hello <arg>" if an argument is provided.

Which one of the following is not a Java feature?

Flag

Previous

Next

Answer

Use of pointers

Portable

Object-oriented

Dynamic and Extensible

The correct answer is: **Use of pointers**

Java does not support the use of pointers like C or C++. Instead, Java uses references to access memory locations. This means that Java automatically manages memory allocation and deallocation, eliminating the need for manual pointer manipulation.

The other options are indeed Java features:

- **Portable**: Java is designed to be platform-independent, allowing Java programs to run on any device that has a Java Virtual Machine (JVM) installed.
- **Object-oriented**: Java is an object-oriented programming language that supports encapsulation, inheritance, and polymorphism.
- **Dynamic and Extensible**: Java is a dynamically-typed language and supports dynamic loading of classes, making it extensible and flexible.

Note that Java does have a concept called "references" which are similar to pointers, but they are managed by the JVM and do not allow direct memory manipulation like pointers in C or C++.

What will be the output of the following Java program?

```
class variable_scope
{
public static void main(String args [])
{
int x;
x = 5;
{
int y = 6;
System.out.print (x + " " + y);
}
System.out.println (x + " " + y);
}
}
```

Flag

Previous

Next

Answer

5 6 5

5 6 5 6

compilation error

Runtime error

It looks like there's a mistake in the code!

The correct answer is: **compilation error**

Here's why:

- The variable `y` is declared and initialized within a block (also known as a scope).
- Once the block ends, the variable `y` goes out of scope and is no longer accessible.
- In the last `println` statement, the code tries to access `y`, which is no longer in scope, resulting in a compilation error.

To fix this, you can declare `y` outside the block, like this:

Java

```java
class variable_scope {
    public static void main(String args[]) {
        int x;
        int y; // declare y here
        x = 5;
        {
            y = 6;
            System.out.print(x + " " + y);
        }
        System.out.println(x + " " + y);
    }
}
```

With this modification, the output will be: **5 6 5 6**

What will be the output of the following Java code?
class output {
public static void main(String args [])
{
double a, b,c ;
a = 3.0/0;
b = 0/4.0;
c=0/0.0;

System.out.println (a);
System.out.println (b);
System.out.println (c);
}
}

Flag

Previous

Next

Answer

Infinity

all of the mentioned

NaN

0.0

The correct answer is: **all of the mentioned**

Here's the output:

- `a = 3.0/0;` will result in **Infinity** because dividing a non-zero number by zero results in infinity in Java.
- `b = 0/4.0;` will result in **0.0** because dividing zero by a non-zero number results in zero.
- `c = 0/0.0;` will result in **NaN (Not a Number)** because dividing zero by zero is undefined in mathematics.

So, the output will be:

```
Infinity
0.0
NaN
```

Note that Java follows the IEEE 754 standard for floating-point arithmetic, which defines the behavior for these special cases.

Which of these keywords is used to define interfaces in Java?

Flag

Previous

Next

Answer

interface

intf

Intf

Interface

The correct answer is: **interface**

In Java, the `interface` keyword is used to define interfaces. An interface is a abstract class that defines a set of methods that must be implemented by any class that implements it.

The other options are not correct:

- **intf** is not a valid Java keyword.
- **Intf** is not a valid Java keyword (note the capital "I").
- **Interface** is not a valid Java keyword (note the capital "I"). Java is case-sensitive, and the correct keyword is `interface` with a lowercase "i".

Example:

Java

```
public interface MyInterface {
    void method1();
    void method2();
}
```

This defines an interface named `MyInterface` with two methods, `method1` and `method2`, that must be implemented by any class that implements this interface.

What is the extension of java code files?

Flag

Previous

Next

Answer

.class

.java

.js

.txt

The correct answer is: **.java**

In Java, the extension of source code files is `.java`. These files contain the Java code that is compiled into bytecode by the Java compiler.

The other options are not correct:

- **.class** is the extension of compiled Java bytecode files, not source code files.
- **.js** is the extension of JavaScript files, which is a different programming language.
- **.txt** is a generic text file extension, not specific to Java.

Example:

```
HelloWorld.java
```

This is a Java source code file that contains the code for a simple "Hello, World!" program. After compilation, the resulting bytecode file would have the same name but with a `.class` extension:

```
HelloWorld.class
```

What is the prototype of the default constructor of this Java class?
public class prototype { }

Flag

Previous

Next

Answer

prototype(void)

public prototype( )

prototype( )

public prototype(void)

The correct answer is: **public prototype()**

In Java, the default constructor (also known as the no-argument constructor) of a class has the same name as the class and no parameters. Its prototype is:

```java
public ClassName()
```

In this case, the class name is `prototype`, so the default constructor prototype is:

```java
public prototype()
```

The other options are not correct:

- **prototype(void)** is not a valid Java constructor prototype. Java does not use the `void` keyword to indicate no parameters.
- **prototype()** is missing the access modifier (`public`).
- **public prototype(void)** is also incorrect because of the `void` keyword.

Note that the default constructor is automatically generated by the Java compiler if no other constructors are defined in the class.

What will be the output of the following Java code?
class conversion
{
public static void main(String args [])
{
double a = 295.04;
int b = 300;
byte c = (byte) a;
byte d = (byte) b;
System.out.println (c + " " + d);
}
}

Flag

Previous

Next

Answer

295 300

295.04 300

39 44

38 43

The correct answer is: **39 44**

Here's what happens in the code:

- `double a = 295.04;` declares a double variable `a` with value `295.04`.
- `int b = 300;` declares an int variable `b` with value `300`.
- `byte c = (byte) a;` casts the double value `a` to a byte. Since a byte can only hold values from -128 to 127, the decimal part is truncated, and the value is narrowed to the range of a byte. `295.04` becomes `39`.
- `byte d = (byte) b;` casts the int value `b` to a byte. Again, the value is narrowed to the range of a byte, so `300` becomes `44`.

Therefore, the output will be:

```
39 44
```

Note that the casting from a larger type (double or int) to a smaller type (byte) results in a loss of precision and a potential change in value.

What is not the use of “this” keyword in Java?

Flag

Previous

Next

Answer

Referring to the instance variable when a local variable has the same name

Passing itself to another method

Passing itself to the method of the same class

Calling another constructor in constructor chaining

The correct answer is: **Passing itself to the method of the same class**

The `this` keyword in Java is used for:

- Referring to the instance variable when a local variable has the same name (e.g., `this.x` to access the instance variable `x` when there's a local variable `x` with the same name)
- Passing itself to another method (e.g., `otherMethod(this)` to pass the current object as an argument)
- Calling another constructor in constructor chaining (e.g., `this(param1, param2)` to call another constructor with different parameters)

However, passing `this` to a method of the same class is not a typical use of the `this` keyword. Instead, you can simply call the method without passing `this` explicitly.

Example:

Java

```
public class MyClass {
    public void method1() {
        method2(); // no need to pass 'this'
    }

    public void method2() {
        // ...
    }
}
```

In this example, `method1` can call `method2` without passing `this` explicitly.

What will be the error in the following Java code?
byte b = 50;
b = b * 50;

Flag

Previous

Next

Answer

b cannot contain value 100, limited by its range

b cannot contain value 50

* operator has converted b * 50 into int, which can not be converted to byte without casting

No error in this code

The correct answer is: *** operator has converted b * 50 into int, which cannot be converted to byte without casting**

In Java, when you multiply a byte value with another value, the result is automatically promoted to an int. So, `b * 50` becomes an int value, which cannot be assigned back to a byte variable without explicit casting.

The issue is that the result of `b * 50` is `2500`, which is an int value, and you're trying to assign it to a byte variable `b`, which can only hold values from -128 to 127.

To fix this, you need to cast the result back to a byte:

Java

```
b = (byte) (b * 50);
```

However, keep in mind that this will result in a loss of precision, as the value `2500` is outside the range of a byte.

The other options are not correct:

- `b` can contain the value `50`, as it's within the range of a byte.
- The range of a byte is not the issue here, but rather the implicit conversion to an int during multiplication.

What will be the output of the following Java code?

```
class char_increment
{
public static void main(String args [])
{
char c1 = 'D';
char c2 = 84;
c2++;
c1++;
System.out.println (c1 + " " + c2);
}
}
```

Flag

Previous

Next

Answer

V E

U F

E U

U E

The correct answer is: **E 85**

Here's what happens in the code:

- `char c1 = 'D';` declares a char variable `c1` with value `D` (which has ASCII value 68).
- `char c2 = 84;` declares a char variable `c2` with value `84` (which is the ASCII value of `T`).
- `c2++;` increments `c2` by 1, so its value becomes `85` (which is not a printable ASCII character).
- `c1++;` increments `c1` by 1, so its value becomes `E` (which has ASCII value 69).

Therefore, the output will be:

```
E 85
```

Note that `c2` is incremented to `85`, which is not a printable ASCII character, so you'll see the numeric value `85` in the output.

The other options are not correct:

- `V E` is incorrect because `c2` is incremented to `85`, not `E`.
- `U F` is incorrect because `c1` is incremented to `E`, not `U`, and `c2` is incremented to `85`, not `F`.
- `E U` is incorrect because `c2` is incremented to `85`, not `U`.

What will be the output of the following Java program?
class Output
{
public static void main(String args [])

```
{
int arr [] = {1, 2, 3, 4, 5};
for ( int i = 0; i < arr.length - 2; ++ i )
System.out.println ( arr [ i ] + " ");
}
}
```

Flag

Previous

Next

Answer

1 2 3

1 2

1 2 3 4 5

1 2 3 4

The correct answer is: **1 2 3**

Here's what happens in the code:

- `int arr[] = {1, 2, 3, 4, 5};` declares an integer array `arr` with 5 elements.
- The `for` loop iterates from `i = 0` to `i < arr.length - 2`, which means it will iterate until `i` is less than `3` (since `arr.length` is `5`).
- Inside the loop, `System.out.println(arr[i] + " ");` prints the current element `arr[i]` followed by a space.

So, the loop will iterate 3 times, printing the first 3 elements of the array:

```
1
2
3
```

The other options are not correct:

- `1 2` is incorrect because the loop iterates 3 times, not 2.
- `1 2 3 4 5` is incorrect because the loop only iterates until `i < arr.length - 2`, which means it misses the last 2 elements.

- `1 2 3 4` is incorrect because the loop only iterates until `i < arr.length - 2`, which means it misses the last element.

Which component is used to compile, debug and execute the java programs?

Flag

Previous

Next

Answer

JVM

JDK

JIT

JRE

The correct answer is: **JDK**

The Java Development Kit (JDK) is a software development kit that provides a set of tools for compiling, debugging, and executing Java programs. It includes:

- `javac` (compiler)
- `java` (runtime environment)
- `jdb` (debugger)
- Other tools like `jar`, `javadoc`, etc.

The JDK is the primary component used for developing and running Java programs.

The other options are not correct:

- **JVM (Java Virtual Machine)** is the runtime environment that executes Java bytecode. While it's part of the JDK, it's not the primary component used for compiling, debugging, and executing Java programs.
- **JIT (Just-In-Time) compiler** is a component of the JVM that compiles Java bytecode into native machine code at runtime. While it's related to the JVM, it's not the primary component used for compiling, debugging, and executing Java programs.
- **JRE (Java Runtime Environment)** is a subset of the JDK that includes only the runtime environment (JVM and libraries) and not the development tools. While it's sufficient for

running Java programs, it's not the primary component used for compiling, debugging, and executing Java programs.

What will be the output of the following Java program?

```
class leftshift_operator
{
public static void main(String args [])
{
byte x = 64;
int i ;
byte y;
i = x << 2;
y = (byte) (x << 2)
System.out.print ( i + " " + y);
}
}
```

Flag

Previous

Next

Answer

0 256

64 0

256 0

0 64

The correct answer is: **256 0**

Here's what happens in the code:

- `byte x = 64;` declares a byte variable `x` with value `64`.
- `i = x << 2;` shifts the bits of `x` 2 places to the left, effectively multiplying `x` by 4. Since `i` is an int, the result is stored in `i` as `256`.
- `y = (byte) (x << 2);` shifts the bits of `x` 2 places to the left, but since `y` is a byte, the result is truncated to a byte. The decimal value `256` is outside the range of a byte, so it wraps around to `0`.

Therefore, the output will be:

```
256 0
```

The other options are not correct:

- `0 256` is incorrect because `y` is truncated to `0`, not `256`.
- `64 0` is incorrect because `i` is multiplied by 4 to become `256`, not `64`.
- `0 64` is incorrect because `y` is truncated to `0`, not `64`.

What will be the output of the following Java code?

```
class String_demo
{
public static void main(String args [])
{
char chars[] = {'a', 'b', 'c'};
String s = new String(chars);
System.out.println (s);
}
}
```

Flag

Previous

Next

Answer

c

b

abc

a

The correct answer is: **abc**

Here's what happens in the code:

- `char chars[] = {'a', 'b', 'c'};` declares a character array `chars` with elements `a`, `b`, and `c`.
- `String s = new String(chars);` creates a new String object `s` from the character array `chars`. The String constructor copies the characters from the array to create a new string.

- `System.out.println(s);` prints the string `s` to the console.

Since the string `s` is created from the character array `chars`, which contains the elements `a`, `b`, and `c`, the output will be:

```
abc
```

The other options are not correct:

- `c` is incorrect because only the last character of the array is not printed.
- `b` is incorrect because only the middle character of the array is not printed.
- `a` is incorrect because only the first character of the array is not printed.

What is Truncation is Java?

Flag

Previous

Next

Answer

Integer value assigned to floating type

Floating-point value assigned to an integer type

Floating-point value assigned to an Floating type

Integer value assigned to floating type

The correct answer is: **Floating-point value assigned to an integer type**

Truncation in Java refers to the loss of precision that occurs when a floating-point value (such as a `double` or `float`) is assigned to an integer type (such as an `int` or `long`). This happens because the decimal part of the floating-point value is discarded, leaving only the integer part.

For example:

Java

```
double d = 3.14;
int i = (int) d;  // truncation occurs here
System.out.println(i);  // prints 3
```

In this example, the decimal part `.14` is truncated, leaving only the integer part `3`.

The other options are not correct:

- **Integer value assigned to floating type**: This is a widening primitive conversion, which does not involve truncation.
- **Floating-point value assigned to a Floating type**: This is not a conversion that involves truncation.
- **Integer value assigned to floating type**: This is a widening primitive conversion, which does not involve truncation.

What will be the output of the following Java code?

```
class A
{
int i ;
void display()
{
System.out.println ( i );
}
}
class B extends A
{
int j;
void display()
{
System.out.println (j);
}
}
class method_overriding
{
public static void main(String args [])
{
B obj = new B();
obj.i =1;
obj.j =2;
obj.display ();
}
}
```

Flag

Previous

Next

Answer

Error

2

0

1

The correct answer is: **2**

Here's what happens in the code:

- Class `A` has an integer field `i` and a method `display()` that prints the value of `i`.
- Class `B` extends class `A`, has an additional integer field `j`, and overrides the `display()` method to print the value of `j`.
- In the `main()` method, an object `obj` of class `B` is created.
- The fields `i` and `j` of `obj` are assigned values `1` and `2`, respectively.
- The `display()` method is called on `obj`, which invokes the overridden method in class `B`, printing the value of `j`, which is `2`.

So, the output will be:

```
2
```

The other options are not correct:

- **Error**: There is no error in the code.
- **0**: The value of `j` is `2`, not `0`.
- **1**: The value of `i` is not printed because the `display()` method in class `B` overrides the one in class `A` and prints the value of `j`.

Which statement is true about Java?

Flag

Previous

Next

Answer

Java is a sequence-dependent programming language

Java is a platform-dependent programming language

Java is a code dependent programming language

Java is a platform independent programming language

The correct answer is: **Java is a platform-independent programming language**

Java is designed to be platform-independent, meaning that Java code can run on any device that has a Java Virtual Machine (JVM) installed, regardless of the underlying operating system or hardware architecture. This is achieved through the use of the JVM, which interprets the Java bytecode and translates it into machine-specific code at runtime.

The other options are not correct:

- **Java is a sequence-dependent programming language**: Java is an object-oriented programming language, not a sequence-dependent programming language.
- **Java is a platform-dependent programming language**: This is the opposite of the correct answer. Java is specifically designed to be platform-independent.
- **Java is a code-dependent programming language**: This statement doesn't make sense in the context of programming languages. Java is a programming language that is independent of the underlying platform, not dependent on the code itself.

What will be the output of the following Java program?

```
class recursion
{
int func ( int n)
{
int result;
if (n == 1)
return 1;
result = func (n - 1);
return result;
}
}
class Output
{
public static void main(String args [])
{
recursion obj = new recursion() ;
System.out.print ( obj.func (5));
}
}
```

Flag

Previous

Next

Answer

1

0

None of the mentioned

120

The correct answer is: **1**

Here's what happens in the code:

- The `func` method in the `recursion` class is a recursive method that takes an integer `n` as input.
- If `n` is 1, the method returns 1.
- If `n` is not 1, the method calls itself with `n-1` as input, and returns the result.
- In the `main` method, an object `obj` of the `recursion` class is created, and the `func` method is called with input `5`.
- The recursive calls will be:
  - `func(5)` calls `func(4)`
  - `func(4)` calls `func(3)`
  - `func(3)` calls `func(2)`
  - `func(2)` calls `func(1)`
  - `func(1)` returns 1
- The final result is 1, which is printed to the console.

The other options are not correct:

- **0**: The method does not return 0 for any input.
- **None of the mentioned**: The method does return a value, which is 1.
- **120**: The method does not calculate factorials, so this value is not correct.

Which of these cannot be used for a variable name in Java?

Flag

Previous

Next

Answer

none of the mentioned

identifier & keyword

keyword

identifier

The correct answer is: **keyword**

In Java, a keyword is a reserved word that has a specific meaning in the language, and it cannot be used as a variable name. Examples of keywords include `public`, `class`, `void`, `int`, etc.

The other options are not correct:

- **none of the mentioned**: This is incorrect because keywords cannot be used as variable names.
- **identifier & keyword**: This is incorrect because an identifier is a valid variable name, but a keyword is not.
- **identifier**: This is incorrect because an identifier is a valid variable name in Java.

Examples of valid variable names in Java include:

- `x`
- `myVariable`
- `_identifier`
- `$dollarSign`

Examples of invalid variable names in Java include:

- `public`
- `class`
- `void`
- `int`

Which of the following is a type of polymorphism in Java Programming?

Flag

Previous

Next

Answer

Execution time polymorphism

Multilevel polymorphism

Compile time polymorphism

Multiple polymorphism

The correct answers are:

- **Compile time polymorphism**
- **Execution time polymorphism**

In Java, there are two types of polymorphism:

1. **Compile-time polymorphism**: Also known as static polymorphism, this occurs when the method to be invoked is determined at compile time. Method overloading is an example of compile-time polymorphism.
2. **Execution time polymorphism**: Also known as dynamic polymorphism, this occurs when the method to be invoked is determined at runtime. Method overriding is an example of execution time polymorphism.

The other options are not correct:

- **Multilevel polymorphism**: This is not a recognized term in Java programming.
- **Multiple polymorphism**: This is not a recognized term in Java programming.

Note that polymorphism is the ability of an object to take on multiple forms, and Java supports both compile-time and execution time polymorphism through method overloading and overriding, respectively.

What will be the output of the following Java program?

```
class evaluate
{
public static void main(String args [])
{
```

```
int arr [] = new int [] {0 , 1, 2, 3, 4, 5, 6, 7, 8, 9};
int n = 6;
n = arr [ arr [n] / 2];
System.out.println ( arr [n] / 2);
}
}
```

Flag

Previous

Next

Answer

2

1

0

4

Clever program!

Let's break it down step by step:

1. `int arr[] = new int[] {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};` creates an integer array `arr` with values from 0 to 9.
2. `int n = 6;` initializes `n` to 6.
3. `n = arr[arr[n] / 2];` is where things get interesting:
    - `arr[n]` is `arr[6]`, which is 6.
    - `arr[n] / 2` is 6 / 2, which is 3.
    - `arr[3]` is 3.
    - So, `n` becomes 3.
4. `System.out.println(arr[n] / 2);` prints the value of `arr[3] / 2`, which is:
    - `arr[3]` is 3.
    - `3 / 2` is 1.

Therefore, the output of the program is:

1

The other options are not correct:

- **2**: This is not the correct output.
- **0**: This is not the correct output.
- **4**: This is not the correct output.

Which of the following is not an OOPS concept in Java?

Flag

Previous

Next

Answer

Encapsulation

Compilation

Inheritance

Polymorphism

The correct answer is: **Compilation**

Compilation is not an OOPS (Object-Oriented Programming System) concept in Java. Compilation is the process of converting source code into machine code, which is a separate step from the OOPS concepts.

The other options are all OOPS concepts in Java:

- **Encapsulation**: The concept of hiding data and methods within an object, and only exposing necessary information through public methods.
- **Inheritance**: The concept of creating a new class based on an existing class, inheriting its properties and methods.
- **Polymorphism**: The concept of an object taking on multiple forms, either through method overloading or method overriding.

OOPS concepts in Java include:

- Encapsulation
- Inheritance
- Polymorphism

- Abstraction

Note: Compilation is an important step in the Java development process, but it's not an OOPS concept.

Which exception is thrown when java is out of memory?

Flag

Previous

Next

Answer

MemoryOutOfBoundsException

MemoryFullException

MemoryError

OutOfMemoryError

The correct answer is: **OutOfMemoryError**

`OutOfMemoryError` is a subclass of `VirtualMachineError` and is thrown when the Java Virtual Machine (JVM) is unable to allocate an object because it is out of memory, and no more memory can be made available by the garbage collector.

The other options are not correct:

- **MemoryOutOfBoundsException**: This is not a valid Java exception.
- **MemoryFullException**: This is not a valid Java exception.
- **MemoryError**: This is a superclass of `OutOfMemoryError`, but it's not the specific exception thrown when Java is out of memory.

Note: `OutOfMemoryError` is a serious error that indicates a problem with the application's memory usage, and it's usually difficult to recover from it.